

07 API - Identity - Security deel 2

📅 09-09-2026 ⌚ Ongeveer 11 minuten 🗨️ Ongeveer 1643 woorden

Je hebt nu Identity achter de rug met seeding, registreren, inloggen en READ-operaties. Je hebt geleerd om JWTBearer en Swagger te implementeren en token authenticatie toe te passen. In dit hoofdstuk wordt Authorisatie aan de hand van Rollen uitgelegd.

Rollen spelen een belangrijk rol in Identity. Het is niet voldoende dat een gebruiker toegang heeft tot je applicatie (Authenticatie). Je wilt ook bepalen welke rechten de gebruiker heeft (Authorisatie). Welke methoden/acties mag de gebruiker uitvoeren. In dit hoofdstuk leer je CUD-operaties toe te passen met de userManager en de roleManager. Om bepaalde standaard-rollen aan te maken wordt vaak een seedingklasse gebruikt.

SeedingKlasse upgraden

In onderstaande code krijg je nu ook de roleManager mee als argument, in de methode `IdentitySeedingAsync` van de seedingklasse (regel 1). In `program.cs` is `IdentityRole` al voorzien in de `AddIdentity` -service. Dit is het type dat gebruikt wordt om de rol te definiëren. Daarna kun je controleren of een rol al bestaat (regel 6 en regel 10) en aanmaken als de rol niet bestaat (regel 7 en regel 11). Als je beslist om een standaardgebruiker aan te maken via je Seedingklasse, kun je nu ineens een rol toekennen (regels 23 en 24). Belangrijk is om na te denken of het nuttig is om een standaardgebruiker aan te maken. Als je beslist om dit te implementeren, moet je er aan denken dat deze gebruiker bij een eerste aanmelding zijn wachtwoord moet wijzigen!

```
1 public async Task IdentitySeedingAsync(UserManager<CustomUser>           CS
2 userManager, RoleManager<IdentityRole> roleManager) {
3     try
4     {
5         // Rollen aanmaken
6         // User
7         bool role = await roleManager.RoleExistsAsync("user");
8         if (!role) await roleManager.CreateAsync(new
9 IdentityRole("user"));
10
11         // Admin
12         role = await roleManager.RoleExistsAsync("admin");
13         if (!role) await roleManager.CreateAsync(new
IdentityRole("admin"));
```

```

14
15     if (userManager.FindByNameAsync("Admin").Result == null)
16     {
17         var defaultUser = new CustomUser
18         {
19             UserName = "Admin",
20             Email = "admin@thomasmore.be",
21             EmailConfirmed = true,
22             PhoneNumberConfirmed = true
23         };
24
25         // Gebruiker aanmaken met rol
26         var admin = await userManager.CreateAsync(defaultUser,
27 "t0LTHxzy.v");
28         if (admin.Succeeded &&
29 userManager.FindByNameAsync("Admin").Result != null)
30             await userManager.AddToRoleAsync(defaultUser, "admin");
31     }
32 } catch (DbException ex){throw new
Exception(ex.Message.ToString());}
33 }

```

In Program.cs moet je de RoleManager ophalen uit de ServiceProvider en meegeven als argument bij het aanroepen van je SeedingKlasse.

```

1     var seeder =
2     scope.ServiceProvider.GetRequiredService<IdentitySeeding>();
3     UserManager<CustomUser> userManager =
4     scope.ServiceProvider.GetRequiredService<UserManager<CustomUser>>();
5     // Voeg de rolemanager toe, injecteer deze als parameters naar de
seeding klasse.
6     RoleManager<IdentityRole> roleManager =
7     scope.ServiceProvider.GetRequiredService<RoleManager<IdentityRole>>();
8     await seeder.IdentitySeedingAsync(userManager, roleManager);

```

GebruikerController

In het vorige hoofdstuk heb je geleerd hoe je met behulp van het **[Authorize]**-attribuut kunt bepalen of een ingelogde gebruiker toegang heeft of niet. Je kunt dit attribuut uitbreiden met rollen. Het is belangrijk dat de naam van de rol precies geschreven is zoals de

naam in de tabel rollen. Als je een typfout maakt, of je gebruikt op een verkeerde plaats hoofdletters, dan krijgt de gebruiker met de juiste rol toch geen toegang.

Rol authorisatie

Hieronder is de rol *admin* toegevoegd aan de *GebruikerController*. Enkel een ingelogde gebruiker met de rol *admin* heeft dus toegang tot de methoden van de *GebruikerController*. De methoden met een **[AllowAnonymous]**-attribuut blijven beschikbaar voor iedereen.

```
1 [Authorize(Roles = "admin")]
2 [Route("api/[controller]")]
3 [ApiController]
4 public class GebruikerController : ControllerBase
```

CS

Dependency Injection van RoleManager

Zorg dat alle dependencies geïnjecteerd zijn. Nu komt daar dus de *RoleManager* bij. De *UserManager* wordt gebruikt om gebruikers te beheren, de *SignInManager* voor het aanmeldproces en de *RoleManager* voor het beheer van rollen. Een rol toevoegen aan een gebruiker, doe je met behulp van de *UserManager*.

```
1 private UserManager<CustomUser> _userManager;
2 private SignInManager<CustomUser> _signInManager;
3 private RoleManager<IdentityRole> _roleManager;
4
5 public GebruikerController(UserManager<CustomUser> userManager,
6 RoleManager<IdentityRole> roleManager, SignInManager<CustomUser>
7 signInManager)
8 {
9     _userManager = userManager;
10    _signInManager = signInManager;
11    _roleManager = roleManager;
12 }
```

CS

In H6 werd nog geen rol toegekend bij het registreren van een gebruiker. Je kunt instellen dat elke gebruiker die zich registreert via de methode *Register* automatisch de rol *user* krijgt. De seeding user (de eerste) in het systeem krijgt de *admin*-rechten.

```
1  if (result.Succeeded)
2  {
3      await _userManager.AddToRoleAsync(user, "user");
4      return Ok();
5  }
```

CS

Gebruiker met rollen opvragen

In het onderstaande voorbeeld, voeg je een extra methode toe, om een lijst van alle gebruikers op te vragen, inclusief de rollen. Om in swagger niet alle properties te laten zien van de gebruiker, maak je gebruik van Dto's.

Maak een `GebruikerMetRollenDto` aan. Je kunt in de toekomst altijd nog een `Gebruiker-UitgebreidDto` voorzien met meer properties.

```
1  public class GebruikerMetRollenDto
2  {
3      public string Id { get; set; }
4      public string Username { get; set; }
5      public string Email { get; set; }
6      public IList<string> Roles { get; set; }
7  }
```

CS

Je haalt een gebruiker op, je voorziet een lijst met code van hierboven. Tijdens deze loop, haal je de rol op van die gebruiker, deze gegevens worden toegevoegd aan die lijst, maar **hier gebruik je de klasse `BeperkteUser`**.

In `GebruikerController.cs`

```
1  [HttpPost("GetAlleUsersMetRollen")]
2  public async Task<IActionResult> GetAlleUsersMetRollen()
3  {
4      var users = await _userManager.Users.ToListAsync();
5
6      var userWithRolesList = new List<GebruikerMetRollenDto>();
7
8      foreach (var user in users)
9      {
10         var roles = await _userManager.GetRolesAsync(user);
11         userWithRolesList.Add(new GebruikerMetRollenDto
12         {
```

CS

```
13         Id = user.Id,  
14         Email = user.Email,  
15         UserName = user.UserName,  
16         Roles = roles  
17     });  
18 }  
19  
20     return Ok(userWithRolesList);  
21 }
```

Resultaat:

Curl

```
curl -X 'POST' \
'https://localhost:44359/api/Gebruiker/GetAlleUsersMetRollen' \
-H 'accept: */*' \
-H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJodHRwOiBoZCZhbmVkbWlucy54bWxzY2FwLnR5cy93cyBhdDA1LzA1LTkzMjN5OXRSL2NsYWltcy9uYnVlaWRlbnpZmllci1iG1jvZWQe4ZDE4LTg0OTNGE0S04Zjc0LFoiMThtdGlZamM=' \
-d ''
```

Request URL

```
https://localhost:44359/api/Gebruiker/GetAlleUsersMetRollen
```

Server response

Code

Details

200

Response body

{
 "users": [
 {
 "id": "76a077c8-109c-4168-a28-23b0fbc573dd",
 "userName": "user@example.com",
 "email": "user@example.com"
 },
 {
 "roles": [
 "admin"
]
 }
],
 {
 "user": {
 "id": "dc01116f-e595-4df2-bf29-54did2e0e7ao",
 "userName": "dries.hoefkens@thomasmore.be",
 "email": "dries.hoefkens@thomasmore.be"
 },
 "roles": [
 "admin"
]
 },
 {
 "user": {
 "id": "f9d95fec-5bf1-4737-b5a9-108a2f9f92c5",
 "userName": "Admin",
 "email": "superTEAM@thomasmore.be"
 },
 "roles": [
 "admin"
]
 }
}

Download

Response headers

content-type: application/json; charset=utf-8
date: Fri, 13 Sep 2024 12:27:30 GMT
server: Microsoft-IIS/10.0
x-powered-by: ASP.NET

GrantPermissions: Toekennen van Rol aan Gebruiker

Je kan in principe ook meerdere rollen toewijzen aan een gebruiker. Zorg dat je dan bij elke toewijzing eerst controleert of die gebruiker de bestaande rol al heeft of niet! Het toekennen van rechten en rollen is een belangrijke denkoefening die je in de ontwerpfase goed moet maken. Als gebruikers te veel rechten krijgen, kunnen ze foute zaken wijzigen in de applicatie. Ook is het belangrijk om ervoor te zorgen dat er altijd minstens één admin aan je applicatie kan, zeker als je applicatie al in de productiefase zit. Op die manier kun je voorkomen dat bepaalde methoden van je applicatie ontoegankelijk worden voor iedereen.

In het onderstaande voorbeeld kan een gebruiker slechts één rol hebben. Je controleert eerst of de nieuwe rol al toegekend is. Daarna verwijder je de vorige rol van de gebruiker en voeg je de nieuwe rol toe.

```

1 public class GrantPermissionDto
2 {
3     public string Email { get; set; }
4     public string RolNaam { get; set; }
5 }

```

```

1 [HttpPost("GrantPermission")]
2 public async Task<IActionResult> GrantPermission(GrantPermissionDto gpm)
3 {
4     CustomUser? gb = await _userManager.FindByEmailAsync(gpm.Email);
5     // Gebruiker zoeken via email opgegeven
6     IdentityRole? rol = await
7     _roleManager.FindByNameAsync(gpm.RolNaam); // nieuwe rol!
8
9     // Gebruiker bestaat en de nieuwe is ook beschikbaar?
10    if (gb != null && rol != null)
11    {
12        // Huidige rollen opvragen in systeem
13        var huidigeRoles = await _userManager.GetRolesAsync(gb);
14
15        if (huidigeRoles.Any()){
16            var huidigeRole = huidigeRoles.First(); // Slechts
17            een rol
18
19            var removeResult = await
20            _userManager.RemoveFromRoleAsync(gb, huidigeRole);
21            if (!removeResult.Succeeded)
22            {
23                if (removeResult.Errors.Count() > 0)
24                {
25                    foreach (var error in removeResult.Errors)
26                        ModelState.AddModelError("message",
27                        error.Description);
28                }
29                return BadRequest(ModelState);
30            }
31
32            IdentityResult res = await

```

```

33 _userManager.AddToRoleAsync(gb, rol.Name);
34
35         if (res.Succeeded)
36             return Ok();
37         else
38         {
39             foreach (IdentityError error in res.Errors)
40                 ModelState.AddModelError("", error.Description);
41
42             return BadRequest(ModelState);
43         }
44     } else
45     {
46         ModelState.AddModelError("", "De gebruiker bestaat niet.");
47
48         ModelState.AddModelError("message", "Onbekende fout");
49         return Unauthorized(ModelState);
50     }
51 }

```

Resultaat:

Request body

```
{
  "email": "dries.hoefkens@thomasmore.be",
  "rolnaam": "user"
}
```

Rol toekennen (nu Admin -> moet user worden)

Execute

Responses

Curl

```
curl -X 'POST' \
  https://localhost:44359/api/Gebruiker/GrantPermissions' \
  -H 'accept: */*' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIs
  -H 'Content-Type: application/json' \
  -d '{
    "email": "dries.hoefkens@thomasmore.be",
    "rolnaam": "user"
  }'
```

Request URL

https://localhost:44359/api/Gebruiker/GrantPermissions

Server response

Code	Details
200	Response headers

```
Response body
{
  "userName": "user@example.com",
  "email": "user@example.com"
},
{
  "roles": [
    "admin"
  ]
},
{
  "user": {
    "id": "dc0116f-e595-4df2-bf29-54d1d2e8e7a0",
    "userName": "dries.hoefkens@thomasmore.be",
    "email": "dries.hoefkens@thomasmore.be"
  },
  "roles": [
    "user"
  ]
},
{
  "user": {
    "id": "f9d95fec-5bf1-4737-b5a9-108a2f9f92c5",
    "userName": "Admin",
    "email": "superteam@thomasmore.be"
  },
  "roles": [
    "admin"
  ]
}
```

CUD-operaties

Create

Een gebruiker aanmaken, daar heb je eigenlijk onbewust al "een deel" script voor voorzien, het registeren kan hiervoor gebruikt worden. Een kritische opmerking hierbij is: het kan zijn dat je doelen en rechten bij deze Create unieke zijn dan bij registeren. In de oefening zal je dit sowieso ervaren.

Update

In het onderstaande voorbeeld wordt het wijzigen van een gebruiker uitgewerkt. Je maakt eerst een `GebruikerWijzigenDto` aan met de gegevens die de gebruiker kan wijzigen.

```
1 public class GebruikerWijzigenDto
2 {
3     public string Id { get; set; }
4     public string? Naam { get; set; }
5     public DateTime? Geboortedatum { get; set; }
6     public string Username { get; set; }
7     public string Email { get; set; }
8     public string CurrentPassword { get; set; }
9     public string Password { get; set; }
10    public string PhoneNumber { get; set; }
11 }
```

CS

Je kunt `Automapper` gebruiken in plaats van de properties handmatig te mappen. Voeg volgende regel toe in `MapperProfile` :

```
1 CreateMap<GebruikerWijzigenDto, CustomUser>();
```

CS

In de `GebruikerController` voeg je een nieuwe methode toe. Deze ontvangt de `GebruikerWijzigenDto` en de id. Na verificatie of de id's gelijk zijn, zoek je de gebruiker op. Als alle gegevens correct zijn ingegeven, voeg je de gewijzigde properties toe aan de bestaande gebruiker. Daarna controleer je of het wachtwoord ingevuld is en wijzig je het bestaande wachtwoord. Dit stuk kun je overslaan indien je niet je eigen gegevens wijzigt. Daarna probeer je de gegevens te updaten.

CS

```
1 [HttpPut("{id}")]
2 public async Task<IActionResult> Put(string id, GebruikerWijzigenDto
3 userDto)
4 {
5     if (userDto == null || id != userDto.Id) return
6     BadRequest(ModelState);
7
8     CustomUser? gebruiker = await _userManager.FindByIdAsync(id);
9
10    if (gebruiker == null) return NotFound(ModelState);
11
12    if (ModelState.IsValid)
13    {
14        try
15        {
16            _mapper.Map(userDto, gebruiker);
17
18            if (!string.IsNullOrEmpty(userDto.Password))
19            {
20                // Het huidige wachtwoord zou moeten worden doorgegeven
21                door de gebruiker, niet de hash
22                var result = await
23                _userManager.ChangePasswordAsync(gebruiker, userDto.CurrentPassword,
24                userDto.Password);
25
26                if (!result.Succeeded)
27                {
28                    foreach (var error in result.Errors)
29                    {
30                        ModelState.AddModelError("message",
31                        error.Description);
32                    }
33                    return BadRequest(ModelState);
34                }
35            }
36        }
37
38        IdentityResult updateResult = await
39        _userManager.UpdateAsync(gebruiker);
40        if (updateResult.Succeeded)
41            return Ok("De gebruiker is gewijzigd");
42        else
43        {
```

```
43         if (updateResult.Errors.Count() > 0)
44         {
45             foreach (var error in updateResult.Errors)
46                 ModelState.AddModelError("message",
47 error.Description);
48         }
49         return BadRequest(ModelState);
50     }
51 }
    catch (DbUpdateConcurrencyException)
    {
        throw;
    }
}
return BadRequest();
}
```

Delete

```
1 [HttpDelete("{id}")]
2 public async Task<IActionResult> Delete(string id)
3 {
4     CustomUser? gebruiker = await _userManager.FindByIdAsync(id);
5     if (gebruiker != null)
6     {
7         IdentityResult result = await
8 _userManager.DeleteAsync(gebruiker);
9         if (result.Succeeded)
10             return Ok("De gebruiker is succesvol verwijderd.");
11         else
12         {
13             if (result.Errors.Count() > 0)
14             {
15                 foreach (var error in result.Errors)
16                     ModelState.AddModelError("message",
17 error.Description);
18             }
19             return BadRequest(ModelState);
20         }
21     }
    return NotFound();
}
```

BestellingController

Zorg dat de `BestellingController` beveiligd is met `[Authorize]` en overschrijf deze volgende methoden met hun eigen beveiligingseisen.

Voor de onderstaande methode hoef je niet ingelogd te zijn:

```
1 // GET: api/Bestelling CS  
2 [AllowAnonymous]  
3 [HttpGet]  
4 public async Task<ActionResult<IEnumerable<Bestelling>>>  
  AlleBestellingenOphalen()
```

De onderstaande methode eist dat gebruiker is ingelogd én de Rol admin heeft:

```
1 [Authorize(Roles = "admin")] CS  
2 [HttpGet("{id}")]  
3 public async Task<ActionResult<BestellingDto>> BestellingOphalen(int id)
```

Je vereist dat een gebruiker is ingelogd en de rol gebruiker heeft.

```
1 [Authorize(Roles = "user")] CS  
2 [HttpPost]  
3 public async Task<ActionResult<Bestelling>>  
  BestellingToevoegen(Bestelling bestelling)
```

Je vereist dat een gebruiker is ingelogd en de rol gebruiker of admin heeft (als je nog andere rollen zou toevoegen).

```
1 [Authorize(Roles = "user,admin")] CS  
2 [HttpPut("{id}")]  
3 public async Task<IActionResult> BestellingWijzigen(int id, Bestelling  
  bestelling)
```

[Hier vind je het WebApiDemo-project.](#)

[Hier vind je een MAUI-project dat de API aanspreekt](#)

Laatst bijgewerkt:09-09-2026, 08:52

Bijdragers: JorenSynaeveTM